

# Purely Implementational LeetCode Problems: Warm-Up & Core Topics

---

A consolidated checklist of implementational and foundational LeetCode problems covering core syntax, loops, and basic data structures. Ideal for building solid baseline coding readiness before tackling advanced algorithms.

## Warm-Up & Basic Implementation

Focus: Core syntax, straightforward conditions, and writing exactly what the prompt requires.

- [2235. Add Two Integers](#) — *Basic function returns.*
- [412. Fizz Buzz](#) — *The classic conditional implementation.*
- [1108. Defanging an IP Address](#) — *Strictly replacing characters in a string.*
- [1281. Subtract the Product and Sum of Digits of an Integer](#) — *Basic while-loop with modulo (%) and division (/).*
- [1470. Shuffle the Array](#) — *Pure index mapping using a single loop.*
- [1480. Running Sum of 1d Array](#) — *Iterating forward and accumulating state.*
- [1672. Richest Customer Wealth](#) — *A straightforward nested loop to sum up 2D array rows.*

## Numbers (Digits, Primes & Math Simulation)

Focus: Base-10 digit extraction, modulo arithmetic, and basic mathematical simulations.

- [7. Reverse Integer](#) — *Handling integer overflow during digit manipulation.*
- [9. Palindrome Number](#) — *Digit extraction without converting to a string.*
- [204. Count Primes](#) — *Classic implementation of the Sieve of Eratosthenes.*
- [258. Add Digits](#) — *Repeated digit summation.*
- [728. Self Dividing Numbers](#) — *Simulate checking every digit of a number within a range loop.*
- [1342. Number of Steps to Reduce a Number to Zero](#) — *A pure simulation using a while loop and even/odd checks.*
- [1716. Calculate Money in Leetcode Bank](#) — *Simulating daily increments and weekly resets using loops and basic addition.*
- [2614. Prime In Diagonal](#) — *Combines matrix traversal with writing a basic, implementational `isPrime()` helper function.*

## Loops (Fibonacci & State Simulations)

Focus: Maintaining and updating variables accurately across loop iterations.

- [509. Fibonacci Number](#) — *Implementing the exact sequence definition using a loop.*
- [1137. N-th Tribonacci Number](#) — *Maintaining three previous states instead of two inside a loop.*
- [1518. Water Bottles](#) — *A pure while-loop simulation of drinking and exchanging empty bottles.*

## Arrays (Building & Traversing)

Focus: Core traversals, copying elements, and managing multiple indices.

- [26. Remove Duplicates from Sorted Array](#) — *Two-pointer technique for in-place modification.*
- [283. Move Zeroes](#) — *Maintaining relative order while shifting elements.*
- [1431. Kids With the Greatest Number of Candies](#) — *Two strict passes: one to find the max, one to evaluate the boolean condition.*
- [1920. Build Array from Permutation](#) — *Basic 1D array mapping and traversal.*
- [1929. Concatenation of Array](#) — *Allocating an array of double size and writing elements to it.*



## Strings (Parsing & Character Operations)

Focus: Character arrays, parsing sequentially, and frequency counting.

- [3110. Score of a String](#) - *Simple string traversal, checking adjacent characters and using ASCII code differences*
- [125. Valid Palindrome](#) — *String parsing, filtering alphanumeric characters, and ignoring case.*
- [242. Valid Anagram](#) — *Basic hash map or fixed-size array frequency counting.*
- [344. Reverse String](#) — *In-place two-pointer swap.*
- [1528. Shuffle String](#) — *Creating a new character array and placing characters at their explicitly given indices.*
- [1678. Goal Parser Interpretation](#) — *Iterating through a string and appending specific outputs based on current and next characters.*
- [2810. Faulty Keyboard](#) — *Simulating typing on a screen, where hitting 'i' reverses typed characters.*



## Matrices (2D Arrays & Grid Constraints)

Focus: Nested loop traversals, boundary checks, and row/column coordinate operations.

- [73. Set Matrix Zeroes](#) — *Row and column state management.*
- [766. Toeplitz Matrix](#) — *Strict boundary traversal, checking if `matrix[i][j] == matrix[i+1][j+1]` across the grid.*
- [867. Transpose Matrix](#) — *Swapping row and column indices.*
- [1572. Matrix Diagonal Sum](#) — *Using purely mathematical indices to sum the primary and secondary diagonals.*